

Enseignant : BELACEL Madani
Module : Informatique
Niveau : ENS — 1ère année — Département de français
Durée : 1h30
Année universitaire : 2025-2026

Objectifs pédagogiques

- À l'issue de ce TP, l'étudiant est capable de :
- Implémenter une pile (LIFO) en Python avec une liste
 - Implémenter une file (FIFO) en Python avec `collections.deque`
 - Utiliser les opérations `push/pop` et `enqueue/dequeue`
 - Résoudre des problèmes classiques avec piles et files
 - Comprendre l'utilité des structures de données en algorithmique
 - Implémenter une calculatrice en notation polonaise inverse (NPI)

Plan du TP

Phase	Durée	Activité
Accueil et révision	5 min	Analogies LIFO/FIFO
Exercice 1	15 min	Implémentation d'une pile
Exercice 2	10 min	Implémentation d'une file
Exercice 3	20 min	Vérification de parenthèses
Exercice 4	20 min	File d'impression
Exercice 5	20 min	Notation polonaise inverse

Exercice 1 — Implémentation d'une pile (LIFO) (15 min)

Énoncé : Créer un script `pile.py` qui implémente une pile avec une liste Python. Définir les fonctions :

1. `empiler(pile, element)` — ajoute au sommet
2. `depiler(pile)` — retire et retourne le sommet
3. `sommet(pile)` — retourne le sommet sans retirer
4. `taille(pile)` — retourne le nombre d'éléments
5. `afficher(pile)` — affiche le contenu

Tester avec : empiler 10, 20, 30, dépiler une fois, empiler 40, afficher.

```
flowchart LR
    subgraph Pile[Pile LIFO]
        direction TB
        S1["30 ← SOMMET"] --> S2["20"]
```

```

S2 --> S3["10"]
S3 --> S4["FOND"]
end
E[empiler 40] --> P2
subgraph P2[Pile après empiler 40]
direction TB
T1["40 ← SOMMET"] --> T2["20"]
T2 --> T3["10"]
T3 --> T4["FOND"]
end

```

Corrigé :

```

def empiler(pile, element):
    pile.append(element)

def depiler(pile):
    if pile:
        return pile.pop()
    return None

def sommet(pile):
    if pile:
        return pile[-1]
    return None

def taille(pile):
    return len(pile)

def afficher(pile):
    print(f"Pile : {pile} (sommet → {sommet(pile)})")

p = []
empiler(p, 10)
empiler(p, 20)
empiler(p, 30)
print(f"Avant dépilement : {p}")
x = depiler(p)
print(f"Dépilé : {x}")
empiler(p, 40)
afficher(p)
print(f"Taille : {taille(p)}")

```

Exercice 2 — Implémentation d'une file (FIFO) (10 min)

Énoncé : Créer un script `file.py` qui implémente une file avec `collections.deque`. Définir les fonctions :

1. `enfiler(file, element)` — ajoute à la fin
2. `defiler(file)` — retire et retourne le début
3. `tete(file)` — retourne le début sans retirer
4. `afficher(file)` — affiche le contenu

```

flowchart LR
subgraph File[File FIFO]
direction LR
D1["TÊTE → A"] --> D2["B"] --> D3["C"] --> D4["QUEUE"]
end
action["defiler → A sort"] --> F2
subgraph F2[File après défiler]

```

```

direction LR
E1["TÊTE → B"] --> E2["C"] --> E3["QUEUE"]
end

```

Corrigé :

```

from collections import deque

def enfiler(file, element):
    file.append(element)

def defiler(file):
    if file:
        return file.popleft()
    return None

def tete(file):
    if file:
        return file[0]
    return None

def afficher(file):
    print(f"File : {list(file)} (tête → {tete(file)})")

f = deque()
enfiler(f, "A")
enfiler(f, "B")
enfiler(f, "C")
print(f"Avant défiler : {list(f)}")
x = defiler(f)
print(f"Défilé : {x}")
enfiler(f, "D")
afficher(f)

```

Exercice 3 — Vérificateur de parenthèses équilibrées (20 min)

Énoncé : Créer un script `parentheses.py` qui vérifie si une expression mathématique est correctement parenthésée. Le programme doit gérer `()`, `[]`, `{}`.

Exemples :

- $(3 + 2) \times [5 - \{1 + 2\}] \rightarrow \text{Correct}$
- $(3 + 2] \times [5 - \{1 + 2\}) \rightarrow \text{Incorrect}$
- $(\{[]\}) \rightarrow \text{Incorrect}$

```

flowchart TD
A[Expression] --> B[Initialiser pile vide]
B --> C[Parcourir chaque caractère]
C --> D{Caractère ouvrant ?}
D -->|Oui| E[Empiler]
E --> C
D -->|Non| F{Caractère fermant ?}
F -->|Non| C
F -->|Oui| G{Pile vide ?}
G -->|Oui| H[Incorrect]
G -->|Non| I[Dépiler et comparer]
I --> J{Correspond ?}
J -->|Non| H
J -->|Oui| C
C --> K[Fin parcours]
K --> L{Pile vide ?}

```

```
L -->|Oui| M[Correct ✓]
L -->|Non| H
```

Corrigé :

```
def est_correcte(expression):
    """Vérifie le parenthésage d'une expression."""
    correspondance = {"(": ")", "[": "]", "{": "}"}
    pile = []

    for char in expression:
        if char in "([{":
            pile.append(char)
        elif char in ")]}":
            if not pile or pile.pop() != correspondance[char]:
                return False

    return len(pile) == 0

expressions = [
    "(3 + 2) × [5 - {1 + 2}]",
    "(3 + 2) × [5 - {1 + 2})",
    "((1 + 2) × 3)",
    "({[]})"
]

for expr in expressions:
    etat = "✓ Correct" if est_correcte(expr) else "x Incorrect"
    print(f"{etat} {expr}")
```

Exercice 4 — Gestion de file d'impression (20 min)

Énoncé : Créer un script `file_impression.py` qui simule une file d'impression. Chaque document a un nom et un nombre de pages. Le programme :

1. Ajoute des documents à la file
2. « Imprime » chaque document (affiche le nom + pages)
3. Affiche le temps d'attente total et par document (1 page = 2 secondes)

Exemple attendu :

```
Ajouté : Rapport_TP5.pdf (15 pages)
Ajouté : Cours_Architecture.pdf (42 pages)
Ajouté : Devoir_Maison.docx (8 pages)

Document 1 : Rapport_TP5.pdf – 15 pages (temps d'impression : 30s, attente cumulée : 30s)
Document 2 : Cours_Architecture.pdf – 42 pages (temps d'impression : 84s, attente cumulée : 114s)
Document 3 : Devoir_Maison.docx – 8 pages (temps d'impression : 16s, attente cumulée : 130s)

Temps total d'impression : 130s
```

Corrigé :

```
from collections import deque

file_impression = deque()

def ajouter_document(nom, pages):
    """Ajoute un document à la file d'impression."""
    doc = {"nom": nom, "pages": pages}
```

```

file_impression.append(doc)
print(f"Ajouté : {nom} ({pages} pages)")

def imprimer():
    """Simule l'impression de tous les documents."""
    temps_total = 0
    numero = 1
    while file_impression:
        doc = file_impression.popleft()
        temps = doc["pages"] * 2
        temps_total += temps
        print(f"Document {numero} : {doc['nom']} – {doc['pages']} pages "
              f"(temps d'impression : {temps}s, attente cumulée : {temps_total}s)")
        numero += 1
    print(f"\nTemps total d'impression : {temps_total}s")

ajouter_document("Rapport_TP5.pdf", 15)
ajouter_document("Cours_Architecture.pdf", 42)
ajouter_document("Devoir_Maison.docx", 8)
print()
imprimer()

```

Exercice 5 — Notation polonaise inverse (calculatrice) (20 min)

Énoncé : Créer un script `npi.py` qui implémente une calculatrice en notation polonaise inverse (NPI). En NPI, l'opérateur vient après les opérandes : `3 4 +` signifie `3 + 4`. Utiliser une pile pour évaluer l'expression.

Fonctionnalités :

1. Lire une expression NPI (opérandes et opérateurs séparés par des espaces)
2. Utiliser une pile : empiler les nombres, dépiler pour les opérations
3. Gérer `+`, `-`, `*`, `/`
4. Gérer les erreurs (division par zéro, expression invalide)

Exemple attendu :

Expression NPI : `3 4 + 2 *`
 Résultat : 14.0

Explication : `3 4 +` → 7, puis `7 2 *` → 14

Corrigé :

```

def evaluer_npi(expression):
    """Évalue une expression en notation polonaise inverse."""
    pile = []
    elements = expression.strip().split()

    for elem in elements:
        if elem in "+-* /":
            if len(pile) < 2:
                return "Erreur : expression invalide"
            b = pile.pop()
            a = pile.pop()
            if elem == "+":
                pile.append(a + b)
            elif elem == "-":
                pile.append(a - b)
            elif elem == "*":

```

```
pile.append(a * b)
elif elem == "/":
    if b == 0:
        return "Erreur : division par zéro"
    pile.append(a / b)
else:
    try:
        pile.append(float(elem))
    except ValueError:
        return f"Erreur : élément inconnu '{elem}'"

if len(pile) == 1:
    return pile[0]
return "Erreur : expression invalide"

# Tests
expressions = [
    "3 4 + 2 *",
    "10 5 /",
    "5 1 2 + 4 * + 3 -",
    "4 0 /"
]

for expr in expressions:
    resultat = evaluer_npi(expr)
    print(f"{expr} → {resultat}")
```

QCM d'auto-évaluation

1. Que signifie LIFO ?
 - a) Last In, First Out ✓
 - b) Large In, Fast Out
 - c) Last Input, Fast Operation
2. Quelle structure de données utiliser pour une file d'attente à la cantine ?
 - a) Une pile
 - b) Une file ✓
 - c) Un dictionnaire
3. En Python, comment implémenter une pile simplement ?
 - a) Avec un dictionnaire
 - b) Avec une liste et les méthodes `append()`/`pop()` ✓
 - c) Avec un tuple
4. Que retourne `pop()` sur une liste vide ?
 - a) Une erreur (`IndexError`) ✓
 - b) None
 - c) 0
5. Dans la NPI, que vaut $5 \ 2 \ - \ 3 \ *$?
 - a) -1

b) 9 ✓

c) 21

Grille d'évaluation TP 06

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Implémentation pile (LIFO)	20 %	Ne sait pas implémenter	Opérations partiellement correctes	Implémentation
Implémentation file (FIFO)	20 %	Ne sait pas implémenter	Opérations partiellement correctes	Implémentation
Algorithme de parenthésage	20 %	Pas d'algorithme	Algorithme partiel	Algorithme co
Simulation file d'impression	20 %	Pas de simulation	Simulation partielle	Simulation co
Calculatrice NPI	20 %	Pas de calculatrice	NPI partiellement implémentée	NPI fonctionn

Déroulement séance — 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil	5 min	5 min
Exercice 1 — Pile	15 min	20 min
Exercice 2 — File	10 min	30 min
Exercice 3 — Parenthésage	20 min	50 min
Exercice 4 — File d'impression	20 min	70 min
Exercice 5 — Calculatrice NPI	20 min	90 min
Total	90 min	

Note pédagogique ENS

Les piles et les files sont des structures de données fondamentales en informatique. Ce TP les aborde de manière progressive :

- Analogies concrètes : Avant de coder, utiliser des objets physiques. Une pile d'assiettes (LIFO) — on pose et on reprend par le dessus. Une file de cantine (FIFO) — le premier arrivé est le premier servi. Ces métaphores sont essentielles pour des élèves de collège.
- Exercice 3 (parenthésage) : C'est un classique des épreuves de NSI au bac. Montrer comment une simple pile peut résoudre un problème complexe de validation syntaxique. On peut aussi l'étendre à la vérification de balises HTML.
- Exercice 5 (NPI) : La notation polonaise inverse est utilisée historiquement par les calculatrices HP. C'est un excellent exercice de synthèse qui combine pile, parsing et gestion d'erreurs. En classe, on peut d'abord faire calculer les élèves « à la main » avec une pile en papier.
- Prolongement possible : Implémenter une calculatrice graphique avec tkinter qui utilise la NPI en interne — projet motivant pour les élèves intéressés.

Les futurs enseignants pourront réutiliser ces exercices pour enseigner les structures de données de façon ludique et progressive.